

x86 Assembly

Register & BIOS Interrupt Rehberi

16-bit • 32-bit • 64-bit Kapsamlı Cheat Sheet

Bu belge x86 mimarisindeki tüm register (yazmaç) ailelerini, adresleme modlarını, bayrakları, SIMD/FPU birimlerini ve en sık kullanılan BIOS kesmelerini **çok sayıda çalışan kod örneğiyle** detaylı biçimde açıklar.

Kapsam: Genel amaçlı register'lar • adresleme modları • segment/işaretçi register'ları • FLAGS & koşullu atlamalar • MMX/SSE/AVX/x87 • kontrol register'ları • komut referansı • INT 10h/13h/15h/16h/1Ah • tam iki aşamalı önyükleyici • korumalı moda geçiş

Hazırlanma Tarihi: 9 Temmuz 2026

Gerçek Mod / Korumalı Mod / Uzun Mod

İçindekiler

1 Giriş: x86 Mimarisine Genel Bakış	2
1.1 İşlemci Çalışma Modları	2
1.2 Sözdizimi: Intel vs AT&T	2
2 Genel Amaçlı Register'lar	3
2.1 Register Boyut Hiyerarşisi	3
2.2 Her Register'ın Detaylı İşlevi ve Örnekleri	3
2.2.1 RAX / EAX / AX – Accumulator (Biriktirici)	3
2.2.2 RBX / EBX / BX – Base (Taban)	4
2.2.3 RCX / ECX / CX – Counter (Sayaç)	4
2.2.4 RDX / EDX / DX – Data (Veri)	4
2.2.5 RSP & RBP – Yığın (Stack) Yönetimi	4
3 Adresleme Modları	5
4 Segment Register'ları	6
5 İşaretçi ve İndeks Register'ları	7
5.1 SI ve DI ile String İşlemleri	7
6 Bayrak Register'ı (FLAGS / EFLAGS / RFLAGS)	7
6.1 Koşullu Atlama Komutları	8
7 SIMD ve FPU Register'ları	9
8 Kayan Nokta (Floating Point) Örnekleri	9
8.1 x87 FPU – Yığın Tabanlı Model	9
8.2 SSE – Skaler (Tek) Kayan Nokta	10
8.3 Tamsayı Kayan Nokta Dönüşümü	11
8.4 Kayan Nokta Karşılaştırma ve Dallanma	11
8.5 Pratik Örnek: İki Nokta Arası Uzaklık	12
8.6 Pratik Örnek: Bir Dizinin Ortalaması	13
8.7 x87 vs SSE Karşılaştırması	13
9 Kontrol ve Özel Register'lar	14
10 Yaygın Komut Hızlı Referansı	14
11 BIOS Interrupt (Kesme) Sistemi	14
11.1 Kesme Nasıl Çalışır? – Kesme Vektör Tablosu	15
11.2 Önemli BIOS Kesmelerine Genel Bakış	15
12 INT 10h – Video Servisleri (Detaylı)	15
12.1 AH=0Eh – Teletype Modunda Karakter Yazma	15
12.2 AH=00h – Video Modu Ayarlama	16
12.3 AH=02h – İmleç Konumu Ayarlama	17
12.4 AH=09h – Karakteri Renkli ve Tekrarlı Yazma	17
12.5 AH=06h – Ekranı Kaydırma / Temizleme	17
12.6 AH=0Ch – Piksel Çizme (Grafik Modu)	18

13 INT 16h – Klavye Servisleri (Detaylı)	18
13.1 AH=00h – Tuş Bekle ve Oku (Blok Eder)	18
13.2 AH=01h – Tuş Var mı? (Blok Etmez)	19
14 INT 13h – Disk Servisleri (Detaylı)	19
14.1 AH=00h – Disk Sistemini Resetleme	19
14.2 AH=02h – Sektör Okuma (CHS)	20
14.3 AH=42h – Genişletilmiş Okuma (LBA)	21
15 INT 15h, 1Ah, 12h – Diğer Önemli Servisler	21
15.1 INT 15h, EAX=E820h – Bellek Haritası	21
15.2 INT 15h, AX=E801h – Genişletilmiş Bellek Boyutu	22
15.3 INT 15h, AH=88h – Genişletilmiş Bellek (Basit)	23
15.4 INT 15h, AX=2401h – A20 Hattını Açma	23
15.5 INT 15h, AH=86h – Mikrosaniye Cinsinden Bekleme	24
15.6 INT 1Ah, AH=00h – Sistem Saatini Okuma	24
15.7 INT 12h – Taban Bellek Boyutu	24
16 BIOS Örneklerinde Kullanılan Register’ların Açıklaması	24
16.1 INT 10h, AH=0Eh – Teletype Karakter Yazma	25
16.2 INT 10h, AH=00h – Video Modu Ayarlama	25
16.3 INT 10h, AH=02h – İmleç Konumu	25
16.4 INT 10h, AH=09h – Renkli/Tekrarlı Karakter	26
16.5 INT 10h, AH=06h – Ekranı Kaydır/Temizle	26
16.6 INT 10h, AH=0Ch – Piksel Çizme	26
16.7 INT 16h, AH=00h ve AH=01h – Klavye	27
16.8 INT 13h, AH=02h – Diskten Sektör Okuma	27
16.9 INT 13h, AH=42h – LBA Okuma	28
16.10 INT 15h, EAX=E820h – Bellek Haritası	28
16.11 Diğer INT 15h Fonksiyonları	28
16.12 INT 1Ah (Saat) ve INT 12h (Bellek)	29
16.13 Özet: Register’ların BIOS Çağrılarındaki Roller	29
17 Tam Örnek: İki Aşamalı Önyükleyici	29
18 Korumalı Moda Geçiş	31
19 Sayfalama (Paging)	32
19.1 Sayfa Tablosu Hiyerarşisi	32
19.2 32-bit Sayfalamayı Etkinleştirme	32
19.3 Sayfa Tablosu Girdisi (PTE) Bayrakları	33
20 Kesme İşleyici (ISR) Yazma	34
20.1 IDT – Kesme Tanımlayıcı Tablosu	34
20.2 Bir IDT Girdisi Ayarlama	35
20.3 Basit Bir Kesme İşleyicisi	35
20.4 PIC’i Yeniden Haritalama	36
20.5 Klavye Kesme İşleyicisi (IRQ1)	36
21 Hızlı Referans: Register Kullanım Kalıpları	37
21.1 System V AMD64 Çağrı Kuralı (Linux 64-bit)	37

22 Pratik Kod Örnekleri	38
22.1 String Uzunluğu (strlen)	38
22.2 İki String'i Karşılaştırma (strcmp)	39
22.3 Sayıyı Onlu (Decimal) Olarak Ekrana Yazma	39
22.4 Baytı Onaltılık (Hex) Olarak Yazma	40
22.5 Basit Gecikme Döngüsü (Busy-wait)	40
23 Derleme ve Test	41
24 Kapanış Notları	41

1. Giriş: x86 Mimarisine Genel Bakış

x86, Intel'in 1978'de çıkardığı 8086 işlemcisinden gelişerek bugüne ulaşmış bir işlemci mimarisidir. Mimari, geriye dönük uyumluluğu (backward compatibility) sayesinde 45 yıl önceki 16-bit kodları hâlâ çalıştırabilir. Register'lar da bu evrimi yansıtır: her yeni nesil, öncekinin register'larını *genişleterek* üzerine ekleme yapmıştır.

Bilgi

Register (Yazmaç) nedir? İşlemcinin *içinde* bulunan, RAM'den çok daha hızlı erişilen küçük hafıza hücreleridir. İşlemci aritmetik, mantık ve veri taşıma işlemlerini doğrudan register'lar üzerinde yapar. RAM erişimi onlarca nanosaniye sürerken register erişimi tek bir saat çevriminin küçük bir kısmında gerçekleşir. Bu yüzden performans kritik kodlarda "veriyi register'da tutmak" temel bir prensiptir.

1.1 İşlemci Çalışma Modları

Mod	Açıklama	Genişlik
Gerçek Mod	8086 uyumlu, segment:offset adresleme, doğrudan donanım & BIOS erişimi. Açılışta işlemci burada başlar.	16-bit
Korumalı Mod	Sanal bellek, bellek koruması, koruma halkaları (ring 0-3), GDT/IDT tabloları	32-bit
Uzun Mod	64-bit, düz bellek modeli, geniş adres alanı, modern işletim sistemleri	64-bit
Sanal 8086	Korumalı mod içinde 16-bit programları çalıştırma alt modu	16-bit

İpucu

Bilgisayar açıldığında işlemci **daima gerçek modda (16-bit)** başlar. BIOS kesmeleri yalnızca gerçek modda kullanılabilir. Önyükleyici (bootloader) önce BIOS kesmeleriyle çekirdeği belleğe yükler, sonra korumalı/uzun moda geçiş yapar. Geçişten sonra BIOS kesmeleri artık kullanılamaz.

1.2 Sözdizimi: Intel vs AT&T

Bu rehberde **Intel sözdizimi** (NASM tarzı) kullanılır çünkü daha okunaklıdır ve önyükleyici geliştirmede standarttır. Farkı bilmek yararlıdır:

Intel (NASM)	AT&T (GAS)
<code>mov eax, 5</code>	<code>movl \$5, %eax</code>
<code>mov eax, [ebx]</code>	<code>movl (%ebx), %eax</code>
Hedef solda: <code>mov hedef, kaynak</code>	Hedef sağda: <code>mov kaynak, hedef</code>

Intel sözdizimi kuralı

`komut hedef, kaynak` — yani **sol taraf hedeftir**. `mov ax, bx` "bx'i ax'e kopyala" demektir. Köşeli parantez `[ ]` bellek erişimi anlamına gelir: `[ebx]` "ebx'in gösterdiği adresteki değer" demektir.

2. Genel Amaçlı Register'lar

x86'da 64-bit modda 16 adet genel amaçlı register vardır. Bunların 8 tanesi tarihsel köke sahiptir ve isimlerinin baş harfleri işlevlerinden gelir.

2.1 Register Boyut Hiyerarşisi

Aynı fiziksel register, farklı isimlerle farklı bit genişliklerinde kullanılabilir. Örneğin **RAX** 64-bit'in tamamıdır; alt 32 biti **EAX**, alt 16 biti **AX**, **AX**'in üst baytı **AH**, alt baytı **AL**'dir.

RAX – 64 bit (bit 63 ...0)	
(üst 32 bit – ayrı ismi yok)	EAX – 32 bit
AX – 16 bit	
AH (üst 8) AL (alt 8)	

64-bit	32-bit	16-bit	8-bit	Geleneksel Amaç
RAX	EAX	AX	AH/AL	Accumulator – aritmetik, çarpma/bölme, syscall dönüş değeri
RBX	EBX	BX	BH/BL	Base – bellek adreslemede taban işaretçisi
RCX	ECX	CX	CH/CL	Counter – döngü ve string işlemlerinde sayaç
RDX	EDX	DX	DH/DI	Data – G/Ç portları, çarpma/bölmede yüksek bölüm
RSI	ESI	SI	SIL	Source Index – string kopyalamada kaynak
RDI	EDI	DI	DIL	Destination Index – string kopyalamada hedef
RBP	EBP	BP	BPL	Base Pointer – yığın çerçevesi (stack frame) tabanı
RSP	ESP	SP	SPL	Stack Pointer – yığının tepesini gösterir
R8–R15	R8D–R15D	R8W–R15W	R8B–R15B	64-bit modda eklenen 8 ek register (özel amacı yok)

⚠ Dikkat

64-bit modda bir register'ın 32-bit parçasına yazmak (örn. **EAX**), üst 32 biti **otomatik olarak sıfırlar**. Ancak 16-bit (**AX**) veya 8-bit (**AL**) parçasına yazmak üst bitleri **korur**. Örnek: **RAX=0xFFFFFFFFFFFFFFFF** iken `mov eax, 0` yaparsanız **RAX=0** olur; ama `mov ax, 0` yaparsanız **RAX=0xFFFFFFFFFFFF0000** kalır. Bu, çok yaygın bir hata kaynağıdır.

2.2 Her Register'ın Detaylı İşlevi ve Örnekleri

2.2.1 RAX / EAX / AX – Accumulator (Biriktirici)

Aritmetik işlemlerin favori register'ıdır. **MUL** ve **DIV** komutları örtük olarak bu register'ı kullanır. Linux'ta **syscall** numarası buraya yazılır ve dönüş değeri buradan okunur.

```

1 mov eax, 5      ; EAX = 5
2 add eax, 3      ; EAX = 8
3 imul eax, 4     ; EAX = 32 (isaretli carpma)
4 mov al, 0xFF    ; sadece alt bayti degistir
5                ; EAX'in ust 24 biti korunur: EAX = 0x000000FF

```

2.2.2 RBX / EBX / BX – Base (Taban)

Genellikle bir bellek bloğunun/dizinin başlangıç adresini tutmak için kullanılır. Gerçek modda dolaylı adreslemede sık kullanılır.

```
1 mov bx, array      ; BX = dizinin baslangic adresi
2 mov al, [bx]       ; AL = dizinin ilk elemani
3 inc bx             ; sonraki elemana gec
4 mov al, [bx]       ; AL = ikinci eleman
```

2.2.3 RCX / ECX / CX – Counter (Sayaç)

LOOP komutu her çalıştığında **CX/ECX**'i bir azaltır ve sıfır olana kadar döngüye devam eder. REP önekiyle string komutlarında tekrar sayısını, SHL/SHR komutlarında ise kaydırma miktarını belirler.

```
1 mov ecx, 5          ; 5 kez donecek
2 label:
3     ; ... yapılacak is ...
4     loop label      ; ECX--, sifir degilse label'e don
5
6 ; SHL/SHR icin CL kullanimi:
7 mov al, 1
8 mov cl, 4
9 shl al, cl          ; AL = 1 << 4 = 16
```

2.2.4 RDX / EDX / DX – Data (Veri)

Çarpma ve bölmede 64/128-bit sonucun *yüksek* yarısını tutar. Ayrıca G/Ç port numarası **DX**'te tutulur.

32-bit çarpma ve bölme – RDX:RAX çifti

```
1 ; CARPMA: EAX * EBX -> sonuc 64-bit olarak EDX:EAX
2 mov eax, 0x10000000
3 mov ebx, 16
4 mul ebx             ; EDX:EAX = EAX * EBX
5                     ; EDX = ust 32 bit, EAX = alt 32 bit
6
7 ; BOLME: EDX:EAX / EBX -> bolum EAX, kalan EDX
8 mov edx, 0          ; ONEMLI: bolmeden once EDX'i temizle!
9 mov eax, 100
10 mov ebx, 7
11 div ebx            ; EAX = 100/7 = 14, EDX = 100 mod 7 = 2
```

⚠ Dikkat

DIV komutundan önce **EDX** (veya 16-bit'te **DX**) mutlaka temizlenmelidir; aksi halde işlemci temizlenmemiş yüksek biti bölmeye dahil eder ve genellikle "bölme taşması" (divide overflow) kesmesi (INT 0) oluşur. İşaretli bölmede CDQ komutu **EAX**'in işaretini **EDX**'e yayar.

2.2.5 RSP & RBP – Yığın (Stack) Yönetimi

RSP yığının tepesini gösterir; PUSH ile azalır, POP ile artar (yığın yukarıdan aşağıya büyür). **EBP** ise bir fonksiyon içindeki yerel değişkenlere ve parametrelere sabit bir referans noktası sağlar.

</> Standart fonksiyon prologu ve epilogu

```

1 sum:
2     push rbp           ; 1) eski taban isaretcisini sakla
3     mov rbp, rsp       ; 2) yeni cerceve tabani olustur
4     sub rsp, 16        ; 3) 16 bayt yerel degisken alani ayir
5
6     mov [rbp-8], rdi    ; ilk parametreyi yerel degiskene koy
7     mov rax, [rbp-8]
8     add rax, rsi        ; RAX = param1 + param2
9
10    mov rsp, rbp        ; 4) yerel alani geri al
11    pop rbp             ; 5) eski tabani geri yukle
12    ret                 ; 6) fonksiyondan don (donus degeri RAX'te)

```

3. Adresleme Modları

Bir komutun operandına nasıl ulaşacağını "adresleme modu" belirler. x86'nın esnek adresleme modları, dizilere ve yapılara erişimi kolaylaştırır.

Mod	Örnek	Anlamı
Anlık (immediate)	<code>mov ax, 5</code>	Sabit değer doğrudan komutun içinde
Register	<code>mov ax, bx</code>	Değer bir register'dan alınır
Doğrudan (direct)	<code>mov ax, [1234h]</code>	Belirli bir bellek adresi
Register dolaylı	<code>mov ax, [bx]</code>	BX'in gösterdiği adresteki değer
Taban + yer değiştirme	<code>mov ax, [bx+4]</code>	Yapı alanlarına erişim
Taban + indeks	<code>mov ax, [bx+si]</code>	İki register toplamı
Ölçekli indeks (SIB)	<code>mov eax, [ebx+esi*4]</code>	Dizi elemanına erişim (4 baytlık)

</> Ölçekli indeks ile bir int dizisinde gezinme

```

1 ; int32 dizisi[i] elemanina erisim: adres = taban + i*4
2 mov ebx, array    ; EBX = dizinin baslangici
3 mov esi, 3        ; ESI = istenen indeks (i = 3)
4 mov eax, [ebx+esi*4] ; EAX = dizi[3]
5                  ; olcek 4 cunku her eleman 4 bayt

```

i Bilgi

Genel formül: `[taban + indeks*ölçek + yer_değiřtirme]`. Ölçek yalnızca 1, 2, 4 veya 8 olabilir (byte, word, dword, qword eleman boyutlarına karşılık gelir). Bu tek komut, C dilindeki `dizi[i]` ifadesinin doğrudan karşılığıdır.

İpucu

LEA (Load Effective Address) komutu, adresi *hesaplar* ama belleğe gitmez; sadece hesaplanan adresi register'a yazar. Bu yüzden hızlı aritmetik için de kullanılır: `lea eax, [ebx+ebx*4]` aslında $EAX = EBX * 5$ yapar, üstelik bayrakları değiştirmeden!

4. Segment Register'ları

Segment register'ları 16-bit'tir ve gerçek modda belleği 64KB'lık bloklara (segment) böler. Fiziksel adres $\text{segment} \times 16 + \text{offset}$ formülüyle hesaplanır.

Register	Açılım	İşlev
CS	Code Segment	Çalışan kodun bulunduğu segment (IP ile eşleşir)
DS	Data Segment	Varsayılan veri segmenti
SS	Stack Segment	Yığının bulunduğu segment (SP/BP ile eşleşir)
ES	Extra Segment	Ek veri segmenti (string hedefi için DI ile)
FS	–	Ek segment (modern OS'lerde thread-local storage)
GS	–	Ek segment (64-bit'te çekirdek veri yapıları)

Fiziksel Adres Hesabı ve segment kurulumu

DS = 0x1000, offset = 0x0234 ise

Fiziksel adres = $0x1000 \times 16 + 0x0234 = 0x10000 + 0x0234 = 0x10234$

```

1 ; Onyukleyicide segmentleri guvenli bir sekilde ayarlama
2 xor ax, ax          ; AX = 0
3 mov ds, ax          ; DS = 0 (segmentlere dogrudan sabit yazilamaz,
4 mov es, ax          ; ES = 0 once bir register'a koymak gerekir)
5 mov ss, ax          ; SS = 0
6 mov sp, 0x7C00      ; yigin onyukleyicinin hemen altinda

```

Bilgi

Korumalı ve uzun modda segment register'ları artık bellek bölme için değil, **segment seçici (selector)** olarak kullanılır ve GDT/LDT tablolarındaki tanımlayıcıları işaret eder. 64-bit'te FS ve GS dışındaki segmentlerin tabanı genellikle 0 kabul edilir (flat / düz model).

5. İşaretçi ve İndeks Register'ları

Register	Ad	İşlev
IP / EIP / RIP	Instruction Pointer	Bir sonraki çalıştırılacak komutun adresi
SP / ESP / RSP	Stack Pointer	Yığının tepesi
BP / EBP / RBP	Base Pointer	Yığın çerçevesi tabanı
SI / ESI / RSI	Source Index	String/dizi kaynağı
DI / EDI / RDI	Destination Index	String/dizi hedefi

5.1 SI ve DI ile String İşlemleri

SI ve DI, MOVSB, LODSB, STOSB gibi string komutlarında otomatik olarak kullanılır. Bu komutlar DF (Direction Flag) bayrağına göre indeksleri otomatik artırır/azaltır.

</> Bir bellek bloğunu kopyalama (memcpy)

```

1 cld                ; DF=0 -> SI ve DI artacak (ileri yon)
2 mov si, source    ; DS:SI = kaynak adres
3 mov di, dest      ; ES:DI = hedef adres
4 mov cx, 100       ; 100 bayt kopyalanacak
5 rep movsb        ; CX kez: [ES:DI] <- [DS:SI], SI++, DI++, CX--

```

Tek satırlık rep movsb ile 100 baytlık kopyalama tamamlanır. Bu, C'deki memcpy(hedef, kaynak, 100) ile eşdeğerdir.

</> Bir alanı sabit değerle doldurma (memset)

```

1 cld
2 mov di, buffer    ; ES:DI = doldurulacak alan
3 mov al, 0         ; doldurulacak deger (0)
4 mov cx, 256       ; 256 bayt
5 rep stosb        ; CX kez: [ES:DI] <- AL, DI++, CX--

```

⚠ Dikkat

RIP (Instruction Pointer) doğrudan mov ile değiştirilemez. Yalnızca jmp, call, ret gibi akış komutlarıyla dolaylı olarak değişir. 64-bit modda RIP-görelî adresleme (RIP-relative), konumdan bağımsız kod (PIC) yazmak için kullanılır.

6. Bayrak Register'ı (FLAGS / EFLAGS / RFLAGS)

Bayrak register'ı, işlemcinin durum bilgilerini tutan tekil bitlerden oluşur. Aritmetik/mantık komutları bu bayrakları günceller; koşullu atlamalar (JE, JG, vb.) bunları okuyarak karar verir.

Bit	Kısa	Ad	Anlamı
0	CF	Carry Flag	Elde/ödünç oluştuğunda 1 (işaretsiz taşma)
2	PF	Parity Flag	Sonucun düşük baytında çift sayıda 1 biti varsa set
4	AF	Auxiliary Carry	BCD aritmetiğinde yarım-bayt eldesi
6	ZF	Zero Flag	Sonuç sıfır ise 1
7	SF	Sign Flag	Sonucun en yüksek biti (negatiflik)
8	TF	Trap Flag	Tek adım (single-step) hata ayıklama modu
9	IF	Interrupt Enable	Donanım kesmeleri açık (1) / kapalı (0)
10	DF	Direction Flag	String işlemleri yönü: 0 artan, 1 azalan
11	OF	Overflow Flag	İşaretili aritmetikte taşma oluştuğunda 1

6.1 Koşullu Atlama Komutları

Karşılaştırmadan (CMP veya TEST) sonra kullanılan atlama komutları. İşaretili ve işaretsiz sayılar için ayrı komutlar olduğuna dikkat edin.

Komut	Koşul	Bayrak	Tür
JE / JZ	Eşit / Sıfır	ZF=1	–
JNE / JNZ	Eşit değil	ZF=0	–
JG / JNLE	Büyük	ZF=0 ve SF=OF	işaretili
JL / JNGE	Küçük	SF≠OF	işaretili
JGE / JNL	Büyük eşit	SF=OF	işaretili
JLE / JNG	Küçük eşit	ZF=1 veya SF≠OF	işaretili
JA / JNBE	Büyük (above)	CF=0 ve ZF=0	işaretsiz
JB / JC	Küçük (below)	CF=1	işaretsiz
JAE / JNC	Büyük eşit	CF=0	işaretsiz
JBE / JNA	Küçük eşit	CF=1 veya ZF=1	işaretsiz

</> Karşılaştırma ve dallanma – basit if/else

```

1 mov eax, [number]
2 cmp eax, 10      ; EAX - 10 hesapla, bayraklari ayarla
3 jg greater      ; isaretili "buyuk mu?" -> EAX > 10 ise atla
4                 ; buraya gelindiye EAX <= 10
5     mov ebx, 0
6     jmp done
7 greater:
8     mov ebx, 1    ; EAX > 10 durumu
9 done:

```

💡 İpucu

TEST eax, eax bir sayının sıfır olup olmadığını kontrol etmenin en hızlı yoludur (cmp eax, 0'dan daha verimli). CLI/STI kesmeleri kapatır/açar; CLD/STD ise string yönünü kontrol eder.

7. SIMD ve FPU Register'ları

Modern işlemciler, tek komutla birden fazla veri üzerinde işlem yapan (SIMD – Single Instruction Multiple Data) özel register setlerine sahiptir. Bunlar grafik, ses, bilimsel hesaplama ve şifrelemede yoğun kullanılır.

Set	Register'lar	Açıklama
x87 FPU	ST0–ST7 (80-bit)	Kayan noktalı (floating point) yığın tabanlı hesaplama
MMX	MM0–MM7 (64-bit)	Tamsayı SIMD (FPU register'larıyla örtüşür)
SSE	XMM0–XMM15 (128-bit)	4×float veya 2×double paralel işlem
AVX	YMM0–YMM15 (256-bit)	8×float paralel işlem
AVX-512	ZMM0–ZMM31 (512-bit)	16×float, maske register'ları (K0–K7)

i Bilgi

XMM, YMM ve ZMM register'ları iç içedir: XMM0, YMM0'ın alt 128 bit'i; YMM0 da ZMM0'ın alt 256 bitidir. Tıpkı AX'in EAX'in içinde olması gibi. Örneğin bir `addps xmm0, xmm1` komutu, iki register'daki 4'er float değeri *aynı anda* toplar.

</> SSE ile 4 float'ı aynı anda toplama

```

1 ; a[4] ve b[4] float dizilerini topla -> sonuc[4]
2 movups xmm0, [a] ; XMM0 = {a0, a1, a2, a3} (128-bit yukle)
3 movups xmm1, [b] ; XMM1 = {b0, b1, b2, b3}
4 addps xmm0, xmm1 ; XMM0 = {a0+b0, a1+b1, a2+b2, a3+b3} tek komutta!
5 movups [result], xmm0

```

Skaler kodda 4 ayrı toplama gerekirken SSE bunu tek komutta yapar – bu *paralellik* SIMD'in gücüdür.

8. Kayan Nokta (Floating Point) Örnekleri

Ondalıkli sayılarla (3.14, -0.5 gibi) işlem yapmak için iki yol vardır: eski **x87 FPU** (yığın tabanlı) ve modern **SSE** (register tabanlı). Bugün derleyiciler neredeyse her zaman SSE kullanır çünkü daha hızlı ve daha basittir; ama x87'yi bilmek eski kodları anlamak için gereklidir.

i Bilgi

Kayan nokta sayıları IEEE 754 standardında saklanır: **tek duyarlık** (float, 32-bit) ve **çift duyarlık** (double, 64-bit). SSE'de `ss` soneki "scalar single" (tek float), `ps` "packed single" (4 float), `sd` ise "scalar double" anlamına gelir. Örneğin `addss` tek float toplar, `addps` dört float'ı paralel toplar.

8.1 x87 FPU – Yığın Tabanlı Model

x87, 8 register'ı (**ST0–ST7**) bir *yığın* gibi kullanır. `fld` (float load) bir sayıyı yığının tepesine (**ST0**) iter; önceki değerler aşağı kayar. İşlemler tepedeki değerler üzerinde yapılır.

</> x87 ile iki double toplama: 3.5 + 2.25

```

1 section .data
2     a    dq 3.5           ; 64-bit double
3     b    dq 2.25
4     result dq 0.0
5
6 section .text
7     finit                ; FPU'yu baslat/sifirla
8     fld qword [a]        ; ST0 = 3.5 (yigin: [3.5])
9     fld qword [b]        ; ST0 = 2.25, ST1 = 3.5 (yigin: [2.25, 3.5])
10    faddp                ; ST1 = ST1+ST0, sonra pop -> ST0 = 5.75
11    fstp qword [result]; result = 5.75, yigindan cek

```

Not

faddp (add and pop) iki tepe değeri toplar ve sonucu yığında tek değer olarak bırakır – böylece yığın taşmasını önler. fstp (store and pop) sonucu belleğe yazıp yığından çıkarır. ”p” soneki hep ”pop” (yığından çek) demektir. İşlem sonunda yığının boş kalması önemlidir.

</> x87 ile bir sayının karekökü: sqrt(2.0)

```

1 section .data
2     x    dq 2.0
3     root dq 0.0
4
5 section .text
6     finit
7     fld qword [x]        ; ST0 = 2.0
8     fsqrt                ; ST0 = sqrt(2.0) = 1.41421...
9     fstp qword [root] ; root = 1.41421

```

8.2 SSE – Skaler (Tek) Kayan Nokta

Modern yöntem. **XMM** register'larını doğrudan kullanır, yığın yoktur. Her işlem tek bir komuttur ve okunması çok daha kolaydır.

</> SSE ile dört temel işlem (tek float)

```

1 section .data
2     a dd 10.0      ; 32-bit float
3     b dd 4.0
4     r dd 0.0
5
6 section .text
7     movss xmm0, [a] ; XMM0 = 10.0 (scalar single yukle)
8     movss xmm1, [b] ; XMM1 = 4.0
9
10    addss xmm0, xmm1 ; XMM0 = 10.0 + 4.0 = 14.0
11    ; subss xmm0, xmm1 -> cikarma
12    ; mulss xmm0, xmm1 -> carpma (40.0)
13    ; divss xmm0, xmm1 -> bolme (2.5)
14
15    movss [r], xmm0 ; sonucu bellege yaz

```

8.3 Tamsayı Kayan Nokta Dönüşümü

Bir tamsayıyı float'a veya float'ı tamsayıya çevirmek özel komutlar gerektirir; doğrudan mov işe yaramaz çünkü bit düzenleri farklıdır.

</> int → float ve float → int dönüşümü

```

1 section .data
2     intVal dd 7      ; tamsayi 7
3     flt dd 3.9      ; float 3.9
4
5 section .text
6     ; --- tamsayi -> float ---
7     cvtsi2ss xmm0, [intVal] ; XMM0 = 7.0 (float olarak)
8
9     ; --- float -> tamsayi (kesme/truncate) ---
10    cvttss2si eax, [flt] ; EAX = 3 (ondalik atilir, YUVARLAMAZ)

```

⚠ Dikkat

cvttss2si (iki 't' ile) ondalık kısmı **atar** (truncate): 3.9 → 3. Tek 't'li cvtss2si ise **yuvarlar**: 3.9 → 4. Karıştırılması çok yaygın bir hatadır. C dilindeki (int) dönüşümü truncate yapar, yani cvttss2si'ye karşılık gelir.

8.4 Kayan Nokta Karşılaştırma ve Dallanma

Float'lar normal cmp ile karşılaştırılamaz. comiss (compare scalar single) iki float'ı karşılaştırıp normal bayrakları (ZF, CF) ayarlar; ardından *işaretsiz* atlama komutları (ja, jb, je) kullanılır.

</> İki float'ı karşılaştırıp büyüğe dallanma

```

1  movss xmm0, [a]
2  movss xmm1, [b]
3  comiss xmm0, xmm1 ; XMM0 ile XMM1'i karsilastir, bayraklari ayarla
4  ja a_greater      ; a > b ise atla (isaretsiz atlama kullan!)
5  jb b_greater      ; a < b ise
6  ; buraya gelindiye esitler

```

💡 İpucu

Kayan nokta karşılaştırmasında **işaretsiz** atlamalar (ja/jb) kullanılır, işaretli olanlar (jg/jl) değil. Bunun nedeni comiss'in sonucu **CF** ve **ZF** bayraklarına yazmasıdır. Ayrıca NaN (Not a Number) değerleri **PF** (parity) bayrağını set eder.

8.5 Pratik Örnek: İki Nokta Arası Uzaklık

Öklid uzaklık formülü $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ – oyunlarda ve grafik programlamada temel bir hesaptır. Tüm kayan nokta kavramlarını birleştirir.

</> SSE ile mesafe hesabı: dx, dy verili

```

1  section .data
2      dx dd 3.0          ; x2 - x1
3      dy dd 4.0          ; y2 - y1
4      distance dd 0.0
5
6  section .text
7  computeDistance:
8      movss xmm0, [dx]    ; XMM0 = dx = 3.0
9      mulss xmm0, xmm0    ; XMM0 = dx*dx = 9.0
10
11     movss xmm1, [dy]    ; XMM1 = dy = 4.0
12     mulss xmm1, xmm1    ; XMM1 = dy*dy = 16.0
13
14     addss xmm0, xmm1    ; XMM0 = 9.0 + 16.0 = 25.0
15     sqrtss xmm0, xmm0   ; XMM0 = sqrt(25.0) = 5.0
16
17     movss [distance], xmm0 ; distance = 5.0
18     ret

```

8.6 Pratik Örnek: Bir Dizinin Ortalaması

</> Float dizisinin ortalamasını hesaplama

```

1 section .data
2     array dd 2.0, 4.0, 6.0, 8.0    ; 4 float
3     count dd 4
4     avg dd 0.0
5
6 section .text
7 average:
8     xorps xmm0, xmm0              ; XMM0 = 0.0 (toplam biriktirici)
9     mov ecx, 4                    ; eleman sayisi
10    mov esi, array                ; ESI = dizi baslangici
11 .loop:
12    movss xmm1, [esi]              ; siradaki elemani al
13    addss xmm0, xmm1              ; toplama ekle
14    add esi, 4                    ; sonraki float (4 bayt)
15    loop .loop                    ; ECX-- , sifir degilse devam
16
17    cvtsi2ss xmm1, [count]         ; XMM1 = 4.0 (adedi float yap)
18    divss xmm0, xmm1              ; XMM0 = toplam / adet = 20.0/4.0 = 5.0
19    movss [avg], xmm0
20    ret

```

Not

`xorps xmm0, xmm0` bir XMM register'ını sıfırlamanın en hızlı yoludur – tıpkı tamsayılarda `xor eax, eax` gibi. Bir register'ı kendisiyle XOR'lamak daima 0 verir. Bu, toplam biriktirici için temiz bir başlangıç sağlar.

8.7 x87 vs SSE Karşılaştırması

x87 FPU	SSE
Yığın tabanlı (ST0-ST7)	Register tabanlı (XMM0-XMM15)
80-bit iç duyarlık	32/64-bit
Karmaşık, yığın yönetimi gerekir	Basit, doğrudan register
Tek değer üzerinde çalışır	Paralel (4× veya 8×) çalışabilir
Eski kod, gömülü sistemler	Modern kod, varsayılan

İpucu

Yeni kod yazıyorsanız **SSE kullanın**. x87 yalnızca eski programları okurken veya 80-bit genişletilmiş duyarlık kesinlikle gerektiğinde tercih edilir. 64-bit modda System V çağrı kuralı zaten float parametreleri **XMM0-XMM7** register'larında geçirir.

9. Kontrol ve Özel Register'lar

Bunlar işletim sistemi seviyesinde, işlemcinin çalışma modunu ve bellek yönetimini kontrol eden ayrıcalıklı register'lardır (yalnızca ring 0).

Register	İşlev
CR0	Korumalı moda geçiş (bit 0 = PE), sayfalama (bit 31 = PG), önbellek kontrolü
CR2	Sayfa hatası (page fault) oluştuğunda erişilmeye çalışılan adres
CR3	Sayfa dizini taban adresi (PDBR) – sanal bellek haritası
CR4	PAE, PSE, SSE, PCID gibi gelişmiş özelliklerin etkinleştirilmesi
CR8	(64-bit) Görev önceliği register'ı (TPR)
GDTR / IDTR	Global / Interrupt tanımlayıcı tablo adres ve boyutları
DR0–DR7	Donanım kesme noktaları (hardware breakpoints) için hata ayıklama register'ları
MSR	Model'e özel register'lar (rdmsr/wrmsr ile erişilir)

10. Yaygın Komut Hızlı Referansı

Komut	İşlev	Komut	İşlev
MOV	Değer taşı	CMP	Karşılaştır (çıkar, bayrak ayarla)
ADD/SUB	Topla / Çıkar	TEST	Bit AND (bayrak ayarla)
INC/DEC	1 artır / azalt	AND/OR/XOR	Bit işlemleri
MUL/IMUL	İşaretsiz / işaretli çarpma	NOT/NEG	Bit tümleyeni / negatif
DIV/IDIV	İşaretsiz / işaretli bölme	SHL/SHR	Sola / sağa kaydırma
PUSH/POP	Yığma at / çek	ROL/ROR	Döndürme (rotate)
CALL/RET	Fonksiyon çağır / dön	LEA	Etkin adres yükle
JMP	Koşulsuz atlama	MOVZX/MOVSX	Sıfır / işaret genişletmeli taşıma
INT	Yazılım kesmesi	NOP	Boş işlem
IN/OUT	G/Ç portundan oku/yaz	HLT	İşlemciyi durdur

Bit işlemleriyle yaygın hileler

```

1 xor eax, eax      ; EAX = 0 (en hızlı sıfırlama yöntemi)
2 test eax, eax     ; EAX sıfır mı? -> ZF ayarlanır
3 and al, 0x0F      ; sadece alt 4 biti tut (maskeleme)
4 or al, 0x80       ; en yüksek biti 1 yap
5 shl eax, 1        ; 2 ile carp (sola 1 kaydır)
6 shr eax, 2        ; 4 e bol (sağa 2 kaydır, isaretsiz)

```

11. BIOS Interrupt (Kesme) Sistemi

BIOS (Basic Input/Output System), anakarttaki bir çip üzerinde bulunan ve donanıma en alt seviyede erişim sağlayan bir yazılımdır. **Kesmeler** (interrupt), donanım hizmetlerine erişmek için standartlaştırılmış giriş noktalarıdır.

11.1 Kesme Nasıl Çalışır? – Kesme Vektör Tablosu

Bir yazılım kesmesi INT n komutuyla tetiklenir. İşlemci, belleğin en başında (0x0000) bulunan **Kesme Vektör Tablosu**'na (IVT) bakar. Bu tabloda her girdi 4 bayttır (2 bayt offset + 2 bayt segment). İşlemci $n \times 4$ adresindeki segment:offset değerine atlar.

Kesme Vektör Tablosu (IVT) – adres 0x0000			
Girdi	Adres	İçerik	Kesme
0	0x0000	offset:segment	INT 0 (bölme hatası)
1	0x0004	offset:segment	INT 1 (tek adım)
...	...	...	...
16	0x0040	offset:segment	INT 10h (video)

i Bilgi

Genel Kullanım Deseni: Kesme numarası hangi *cihazı/hizmeti* seçer. AH register'ı ise o hizmet içindeki hangi *alt fonksiyonun* çağrılacağını belirler. Diğer register'lar (AL, BX, CX, DX) parametreleri taşır. Sonuç genellikle register'larda ve/veya CF (Carry) bayrağında döner: CF=1 çoğunlukla "hata oluştu" anlamına gelir.

11.2 Önemli BIOS Kesmelerine Genel Bakış

INT	Hizmet	Açıklama
10h	Video Servisleri	Ekrana yazı yazma, piksel çizme, mod ayarı, imleç
11h	Ekipman Listesi	Takılı donanımların bit maskesi
12h	Bellek Boyutu	Kullanılabilir taban bellek (KB)
13h	Disk Servisleri	Sektör okuma/yazma, disk resetleme, LBA
14h	Seri Port	RS-232 haberleşme
15h	Sistem Servisleri	Geniştirilmiş bellek haritası (E820), A20, APM
16h	Klavye Servisleri	Tuş okuma, klavye durumu
17h	Yazıcı Servisleri	Paralel port yazıcı
19h	Bootstrap	Yeniden önyükleme
1Ah	Saat Servisleri	Sistem saati, RTC okuma

12. INT 10h – Video Servisleri (Detaylı)

Ekrana ilgili tüm işlemler bu kesme üzerinden yapılır.

12.1 AH=0Eh – Teletype Modunda Karakter Yazma

Bu fonksiyon, imleci otomatik ilerleterek tek bir karakteri ekrana basar. Önyükleyicilerde "Hello World" yazdırmanın standart yoludur.

Giriş**Anlam**

AH = 0Eh	Fonksiyon: teletype yazma
AL = karakter	Yazdırılacak ASCII karakter kodu
BH = sayfa	Görüntü sayfası (genellikle 0)
BL = renk	Grafik modunda ön plan rengi

</> Null ile biten metin yazan yeniden kullanılabilir rutin

```

1 printString:          ; giris: SI = metnin adresi
2   push ax
3   mov ah, 0x0E        ; teletype fonksiyonu
4 .loop:
5   lodsb               ; DS:SI'daki bayti AL'e yukle, SI++
6   test al, al         ; AL == 0 mi? (metin sonu)
7   jz .done
8   int 0x10            ; karakteri ekrana yaz
9   jmp .loop
10 .done:
11   pop ax
12   ret
13
14 ; kullanimi:
15 mov si, message
16 call printString
17 message: db "Merhaba, Assembly Dunyasi!", 0x0D, 0x0A, 0

```

**Satır sonu karakterleri**

0x0D (Carriage Return – satır başı) ve 0x0A (Line Feed – alt satır) birlikte kullanılır. Sadece 0x0A imleci alt satıra indirir ama başa getirmez; bu yüzden ikisi birlikte gerekir.

12.2 AH=00h – Video Modu Ayarlama**AL Mod****Açıklama**

03h	80×25 metin	16 renkli standart metin modu
12h	640×480	16 renkli grafik (VGA)
13h	320×200	256 renkli grafik (oyunların klasiği)

```

1 mov ah, 0x00          ; video modu ayarla
2 mov al, 0x03          ; 80x25 metin modu (ekrani da temizler)
3 int 0x10

```

12.3 AH=02h – İmleç Konumu Ayarlama

Giriş

Anlam

AH=02h, BH=sayfa	İmleç konumu ayarla
DH = satır	0–24 arası (üstten)
DL = sütun	0–79 arası (soldan)

</> İmleci 10. satır, 20. sütuna taşıma

```

1 mov ah, 0x02
2 mov bh, 0      ; sayfa 0
3 mov dh, 10     ; satır 10
4 mov dl, 20     ; sutun 20
5 int 0x10

```

12.4 AH=09h – Karakteri Renkli ve Tekrarlı Yazma

İmleci ilerletmeden, belirtilen karakteri belirtilen renkte ve sayıda yazar. Renkli arayüzler çizmek için idealdir.

Giriş

Anlam

AH=09h, AL=karakter	Yazılacak karakter
BH=sayfa, BL=öznitelik	Renk: alt 4 bit ön plan, üst 4 bit arka plan
CX = tekrar sayısı	Kaç kez yazılacak

</> Beyaz zemin üzerine kırmızı 10 yıldız

```

1 mov ah, 0x09
2 mov al, '*'      ; yildiz karakteri
3 mov bh, 0        ; sayfa 0
4 mov bl, 0xF4     ; F=beyaz arka plan, 4=kırmızı ön plan
5 mov cx, 10       ; 10 kez
6 int 0x10

```

i Bilgi

Renk kodları (4-bit): 0=Siyah, 1=Mavi, 2=Yeşil, 3=Cyan, 4=Kırmızı, 5=Magenta, 6=Kahve, 7=Açık gri, 8=Koyu gri, 9=Açık mavi, A=Açık yeşil, B=Açık cyan, C=Açık kırmızı, D=Açık magenta, E=Sarı, F=Beyaz.

12.5 AH=06h – Ekranı Kaydırma / Temizleme

Bir pencere alanını yukarı kaydırır. AL=0 verildiğinde tüm alanı temizler – ekranı belirli renkle temizlemenin yaygın yolu.

</> Tüm ekranı mavi zeminle temizleme

```

1 mov ah, 0x06      ; yukari kaydir / temizle
2 mov al, 0         ; 0 = tum pencereyi temizle
3 mov bh, 0x1F      ; renk: 1=mavi arka, F=beyaz on plan
4 mov cx, 0x0000     ; sol-ust kose (satir 0, sutun 0)
5 mov dx, 0x184F     ; sag-alt kose (satir 24=0x18, sutun 79=0x4F)
6 int 0x10

```

12.6 AH=0Ch – Piksel Çizme (Grafik Modu)**Giriş****Anlam**

AH=0Ch, AL=renk Renk numarası (0–255)

CX = X koordinatı Sütun

DX = Y koordinatı Satır

</> Grafik modda yatay bir kırmızı çizgi çizme

```

1 mov ah, 0x00      ; once grafik moduna gec
2 mov al, 0x13      ; 320x200, 256 renk
3 int 0x10
4
5 mov ah, 0x0C      ; piksel ciz
6 mov al, 4         ; renk: kirmizi
7 mov dx, 100       ; Y = 100 (sabit satir)
8 mov cx, 0         ; X = 0 den basla
9 .draw:
10 int 0x10          ; (CX, DX) noktasina piksel koy
11 inc cx            ; X++
12 cmp cx, 320       ; ekran genisligine ulastik mi?
13 jl .draw          ; hayirsa devam

```

13. INT 16h – Klavye Servisleri (Detaylı)**13.1 AH=00h – Tuş Bekle ve Oku (Blok Eder)**

Bir tuş basılana kadar programı bekletir. AL'de ASCII kodu, AH'de tarama kodu (scan code) döner.

</> Kullanıcıdan tuş bekleyip ekrana yankılama (echo terminali)

```

1 waitKey:
2     mov ah, 0x00 ; tus bekle ve oku
3     int 0x16     ; AL = basılan tusun ASCII kodu
4
5     cmp al, 0x0D ; Enter'a mi basıldı?
6     je  newLine
7
8     mov ah, 0x0E ; teletype yazma
9     int 0x10     ; okunan tusu ekrana yaz
10    jmp waitKey
11
12 newLine:
13     mov ah, 0x0E
14     mov al, 0x0D
15     int 0x10
16     mov al, 0x0A
17     int 0x10
18     jmp waitKey

```

13.2 AH=01h – Tuş Var mı? (Blok etmez)

Tampon boşsa **ZF=1** döner (tuş yok); doluyrsa **ZF=0** olur ve **AL**'de tuş bilgisi bulunur ama *tampondan silinmez*. Oyun döngülerinde giriş kontrolü için idealdir.

</> Oyun döngüsünde bloke etmeden tuş kontrolü

```

1 gameLoop:
2     ; ... oyun mantigini guncelle, ekrani ciz ...
3
4     mov ah, 0x01 ; tampon dolu mu?
5     int 0x16
6     jz  gameLoop ; ZF=1 -> tus yok, donguye devam
7
8     ; buraya gelindiyse bir tus var, simdi tampondan al
9     mov ah, 0x00
10    int 0x16 ; AL = tus
11    cmp al, 'q' ; cikis tusu mu?
12    jne gameLoop
13    ; q'ya basildi -> oyundan cik

```

14. INT 13h – Disk Servisleri (Detaylı)

Diskten sektör okuma, önyükleyicinin çekirdeği (kernel) belleğe yüklemesi için kritiktir. Adresleme CHS (Cylinder-Head-Sector) veya modern LBA yöntemiyle yapılır.

14.1 AH=00h – Disk Sistemini Resetleme

Okuma hatalarından sonra sürücüyü sıfırlamak için kullanılır. Yeniden deneme mantığının parçasıdır.

```

1 mov ah, 0x00 ; disk sistemini resetle

```

```

2 mov dl, 0x80      ; ilk sabit disk
3 int 0x13

```

14.2 AH=02h – Sektör Okuma (CHS)

Giriş

Anlam

AH=02h	Sektör oku fonksiyonu
AL	Okunacak sektör sayısı
CH	Silindir (cylinder) numarası
CL	Sektör numarası (1'den başlar!)
DH	Kafa (head) numarası
DL	Sürücü (00h=disket, 80h=ilk HDD)
ES:BX	Verinin yükleneceği bellek adresi

Çıkış

Anlam

CF=0	Başarılı, AL=okunan sektör sayısı
CF=1	Hata, AH=hata kodu

</> Yeniden deneme mantığıyla sağlam sektör okuma

```

1 diskRead:      ; ES:BX hedef, sektor bilgileri ayarli varsayilir
2   mov di, 3    ; en fazla 3 kez dene
3 .retry:
4   mov ah, 0x02 ; sektor oku
5   mov al, 1    ; 1 sektor
6   mov ch, 0    ; silindir 0
7   mov cl, 2    ; sektor 2 (1. sektor onyukleyicidir)
8   mov dh, 0    ; kafa 0
9   mov dl, 0x80 ; ilk sabit disk
10  int 0x13
11  jnc .success  ; CF=0 -> okuma tamam
12
13  ; hata olustu: diski resetle ve tekrar dene
14  xor ah, ah    ; AH=0 -> reset fonksiyonu
15  int 0x13
16  dec di
17  jnz .retry    ; deneme hakki kaldiyse tekrar
18  jmp diskError ; 3 deneme de basarisiz
19
20 .success:
21  ret

```

⚠ Dikkat

Sektör numarası (**CL**) **1'den başlar**, 0'dan değil! İlk sektör (LBA 0) CHS'de silindir 0, kafa 0, sektör 1'dir. Ayrıca gerçek diskler ara sıra okuma hatası verir; bu yüzden profesyonel önyükleyiciler daima yeniden deneme (retry) döngüsü kullanır.

14.3 AH=42h – Genişletilmiş Okuma (LBA)

Modern büyük diskler için CHS yetersizdir. LBA (Logical Block Addressing) sektörleri 0'dan başlayan tek bir numarayla adresler. Parametreler bir "Disk Adres Paketi" (DAP) yapısı üzerinden geçer.

⌘ LBA ile sektör okuma ve DAP yapısı

```

1  mov ah, 0x42      ; genişletilmiş okuma
2  mov dl, 0x80      ; disk
3  mov si, dap       ; DS:SI = adres paketi
4  int 0x13
5  jc  diskError
6
7  ; Disk Adres Paketi (16 bayt)
8  dap:
9      db 0x10        ; paket boyutu (16)
10     db 0           ; ayrılmış (0)
11     dw 1           ; okunacak sektör sayısı
12     dw 0x8000      ; hedef offset
13     dw 0x0000      ; hedef segment
14     dq 1           ; başlangıç LBA numarası (64-bit)

```

15. INT 15h, 1Ah, 12h – Diğer Önemli Servisler

15.1 INT 15h, EAX=E820h – Bellek Haritası

Modern işletim sistemlerinin RAM haritasını öğrenmesinin standart yoludur. Her çağrı, bir bellek bölgesini tanımlayan 20/24 baytlık bir yapı döndürür ve **EBX**'i bir sonraki bölge için günceller. **EBX**=0 olunca liste biter.

</> E820 bellek haritası okuma döngüsü

```

1 memoryMap:
2     mov di, buffer      ; ES:DI = sonucun yazilacagi yer
3     xor ebx, ebx        ; ilk cagri icin EBX = 0
4     mov edx, 0x534D4150 ; 'SMAP' imzasi (sabit)
5 .loop:
6     mov eax, 0xE820     ; her cagrida yeniden ayarla
7     mov ecx, 24         ; bir girdinin boyutu (bayt)
8     int 0x15
9     jc .done            ; CF=1 -> hata veya liste sonu
10
11    add di, 24           ; sonraki girdi icin ilerle
12    test ebx, ebx       ; EBX=0 -> son girdiydi
13    jnz .loop
14 .done:
15    ; tampon artik bellek bolgelerinin listesini icerir

```

i Bilgi

Her E820 girdisi şunları içerir: 8 bayt taban adres, 8 bayt uzunluk, 4 bayt tür (1=kullanılabilir RAM, 2=rezerve, 3=ACPI, 4=NVS). İşletim sistemi bu haritayı kullanarak hangi bellek bölgelerini güvenle kullanabileceğini belirler.

15.2 INT 15h, AX=E801h – Genişletilmiş Bellek Boyutu

E820'den daha basit bir alternatiftir. 15 MB üstündeki belleği iki register çiftinde döndürür: 1–16 MB arası ve 16 MB üstü.

Çıkış**Anlam**

AX / CX 1–16 MB arası bellek (1 KB birimlerle)
BX / DX 16 MB üstü bellek (64 KB birimlerle)

</> E801 ile toplam RAM'i hesaplama

```

1  xor cx, cx      ; CX ve BX'i temizle (bazi BIOS'lar AX/BX,
2  xor bx, bx      ; digerleri CX/DX kullanir)
3  mov ax, 0xE801
4  int 0x15
5  jc .error
6  ; Bazi BIOS'lar sonucu AX:BX, bazilari CX:DX doner.
7  ; CX=0 ise AX/BX'i kullan:
8  test cx, cx
9  jnz .cxdx
10 mov [lowMem], ax ; 1-16MB (KB cinsinden)
11 mov [highMem], bx ; 16MB ustu (64KB cinsinden)
12 jmp .end
13 .cxdx:
14 mov [lowMem], cx
15 mov [highMem], dx
16 .end:
17 .error:

```

15.3 INT 15h, AH=88h – Genişletilmiş Bellek (Basit)

En eski ve en basit yöntem. **AX**'te 1 MB üstündeki bellek miktarını KB cinsinden döndürür. Yalnızca 64 MB'a kadar ölçeklenir, bu yüzden modern sistemlerde E820/E801 tercih edilir.

```

1  mov ah, 0x88
2  int 0x15      ; AX = 1MB ustundeki bellek (KB), en fazla ~64MB

```

15.4 INT 15h, AX=2401h – A20 Hattını Açma

A20 adres hattı, 1 MB üstündeki belleğe erişim için açılmalıdır (tarihsel bir 8086 uyumluluk kısıtı). BIOS üzerinden açmak en temiz yoldur.

</> A20 hattını BIOS ile açma ve doğrulama

```

1  mov ax, 0x2403 ; once A20 destegi var mi diye sor
2  int 0x15
3  jc .notSupported
4
5  mov ax, 0x2402 ; A20 mevcut durumunu oku
6  int 0x15
7  cmp al, 1
8  je .alreadyOn ; AL=1 ise zaten aciktir
9
10 mov ax, 0x2401 ; A20 hattini ETKINLESTIR
11 int 0x15
12 jc .error
13 .alreadyOn:
14 .notSupported:
15 .error:

```

i Bilgi

A20 açık değilse, 1 MB sınırının üstündeki adresler "sarılır" (wrap-around) ve 0'dan başlar gibi davranır. Korumalı moda geçmeden **önce** A20'nin açık olması şarttır; aksi halde çekirdeğiniz yüksek bellekte doğru çalışmaz. BIOS yöntemi başarısız olursa alternatifler klavye denetleyicisi (port 0x64) veya "Fast A20" (port 0x92) üzerinden açmaktır.

15.5 INT 15h, AH=86h – Mikrosaniye Cinsinden Bekleme

Belirtilen süre kadar (CX:DX mikrosaniye) programı bekletir. Gecikme oluşturmanın hassas yoludur.

</> Tam olarak 500 milisaniye bekleme

```
1  mov ah, 0x86
2  mov cx, 0x0007      ; CX:DX = 500000 mikrosaniye
3  mov dx, 0xA120      ; (0x0007A120 = 500000)
4  int 0x15            ; ~0.5 saniye bekle
```

15.6 INT 1Ah, AH=00h – Sistem Saatini Okuma

BIOS, saniyede yaklaşık 18.2 kez artan bir sayaç tutar. CX:DX çiftinde geri döner; bu değer basit zamanlama veya rastgele sayı tohumu (random seed) için kullanılabilir.

</> Sistem tik sayacını okuyup gecikme oluşturma

```
1  ; Baslangic zamanini oku
2  mov ah, 0x00
3  int 0x1A          ; CX:DX = gun basindan beri gecen tik sayisi
4  mov [startTick], dx
5
6  wait:
7      mov ah, 0x00
8      int 0x1A
9      mov ax, dx
10     sub ax, [startTick]
11     cmp ax, 18      ; ~18 tik = ~1 saniye
12     jl  wait        ; 1 saniye gecene kadar bekle
```

15.7 INT 12h – Taban Bellek Boyutu

Parametre almaz; AX'te kullanılabilir taban bellek boyutunu KB cinsinden döndürür (tipik olarak 640).

```
1  int 0x12          ; AX = KB cinsinden taban bellek
2  ; AX genellikle 640 (0x280) doner
```

16. BIOS Örneklerinde Kullanılan Register'ların Açıklaması

Bu bölüm, yukarıdaki BIOS kesme örneklerinde geçen her register'ın o çağrıda *neden* kullanıldığını tek tek açıklar. Böylece rehberin ilk yarısındaki register bilgisi ile ikinci yarısındaki kesme örnekleri birbirine bağlanır.

i Bilgi

Hatırlatma – genel desen: Neredeyse tüm BIOS çağrılarında **AH** *fonksiyon seçicisidir* (hangi alt işlemin çalışacağını belirler). **AL**, **BX**, **CX**, **DX** *parametreleri* taşır. Sonuç register'lar da ve/veya **CF** (Carry) bayrağında döner. **ES:BX**, **DS:SI**, **ES:DI** gibi *çiftler* ise bir bellek adresini (segment:offset) gösterir.

16.1 INT 10h, AH=0Eh – Teletype Karakter Yazma

Register	Yön	Bu örnekteki rolü
AH	giriş	0x0E yazılır – "teletype yazma" fonksiyonunu seçer
AL	giriş	Ekrana basılacak karakterin ASCII kodu (lods b her seferinde buraya bir bayt yükler)
SI	giriş	Metnin başlangıç adresi; lods b bunu okuyup otomatik artırır
BH	giriş	Görüntü sayfası (0); genelde önemsizdir

Not

Örnekte lods b komutu şu üç işi tek adımda yapar: **DS:SI**'nin gösterdiği baytı **AL**'ye yükler, sonra **SI**'yi bir artırır. Yani **AL** ve **SI** birlikte çalışır: **SI** "nerede olduğumuzu", **AL** "o an hangi karakteri yazdığımızı" tutar. test al, al ile **AL=0** (metin sonu) kontrol edilir.

16.2 INT 10h, AH=00h – Video Modu Ayarlama

Register	Yön	Bu örnekteki rolü
AH	giriş	0x00 – "video modu ayarla" fonksiyonu
AL	giriş	İstenen mod numarası (örn. 0x13 = 320×200, 256 renk)

Burada yalnızca **AX** kullanılır: **AH** ne yapılacağını, **AL** hangi modun seçileceğini söyler. Çağrı ekranı da temizler.

16.3 INT 10h, AH=02h – İmleç Konumu

Register	Yön	Bu örnekteki rolü
AH	giriş	0x02 – "imleç konumu ayarla"
BH	giriş	Görüntü sayfası (0)
DH	giriş	Hedef satır (0–24) – DX 'in üst baytı
DL	giriş	Hedef sütun (0–79) – DX 'in alt baytı

İpucu

Burada **DH** ve **DL**'nin ayrı ayrı kullanılması, tek bir 16-bit register'ın (**DX**) iki 8-bit parçaya bölünmesinin klasik örneğidir: satır ve sütun tek bir register'da paketlenmiştir. **DH**=satır, **DL**=sütun.

16.4 INT 10h, AH=09h – Renkli/Tekrarlı Karakter

Register	Yön	Bu örnekteki rolü
AH	giriş	0x09 – "renkli karakter yaz"
AL	giriş	Yazılacak karakter ('*')
BH	giriş	Görüntü sayfası (0)
BL	giriş	Renk özniteliği: üst 4 bit arka plan, alt 4 bit ön plan (0xF4 = beyaz zemin/kırmızı)
CX	giriş	Karakterin kaç kez tekrarlanacağı (10)

Burada **CX**'in "sayaç" rolü belirgindir: kaç kopya yazılacağını o tutar. **BL** ise tek bir baytta iki renk bilgisini (ön/arka) paketler.

16.5 INT 10h, AH=06h – Ekranı Kaydır/Temizle

Register	Yön	Bu örnekteki rolü
AH	giriş	0x06 – "yukarı kaydır / temizle"
AL	giriş	Kaydırılacak satır sayısı; 0 = tüm pencereyi temizle
BH	giriş	Yeni boş alanın renk özniteliği (0x1F = mavi/beyaz)
CH/CL	giriş	CX = pencerenin sol-üst köşesi (satır/sütun)
DH/DL	giriş	DX = pencerenin sağ-alt köşesi (0x184F = 24/79)

Burada **CX** ve **DX**, her biri iki 8-bitlik parçaya bölünerek bir dikdörtgenin iki köşesini (4 koordinat) toplam iki register'da taşır.

16.6 INT 10h, AH=0Ch – Piksel Çizme

Register	Yön	Bu örnekteki rolü
AH	giriş	0x0C – "piksel yaz"
AL	giriş	Pikselin renk numarası (0-255)
CX	giriş	X koordinatı (sütun) – döngüde inc cx ile artırılır
DX	giriş	Y koordinatı (satır) – sabit tutulur

Çizgi çizme örneğinde **DX** (Y) sabit kalırken **CX** (X) her turda artırılır; böylece yatay bir çizgi oluşur. Bu, register'ların bir döngü değişkeni olarak nasıl kullanıldığını gösterir.

16.7 INT 16h, AH=00h ve AH=01h – Klavye

Register	Yön	Rolü
AH	giriş	0x00 = tuş bekle, 0x01 = tuş var mı kontrol et
AL	çıkış	Basılan tuşun ASCII kodu (BIOS doldurur)
AH	çıkış	Tuşun tarama kodu (scan code)
ZF	çıkış	(Yalnızca AH=01h) tampon boşsa 1, doluyorsa 0

Not

Bu örnek, register'ların hem *giriş* hem *çıkış* taşıyabildiğini gösterir: çağrıdan önce **AH**'ye fonksiyon numarasını yazarız, çağrıdan sonra *aynı* **AX** register'ının içinde BIOS bize sonucu (**AL**=ASCII, **AH**=scancode) geri verir. AH=01h'de asıl sonuç **ZF** bayrağındadır – bu yüzden çağrıdan hemen sonra jz ile kontrol edilir.

16.8 INT 13h, AH=02h – Diskten Sektör Okuma

Bu, en çok register kullanan örnektir. Her biri CHS adreslemesinin bir parçasını taşır:

Register	Yön	Bu örnekteki rolü
AH	giriş	0x02 – "sektör oku" fonksiyonu
AL	giriş	Kaç sektör okunacağı (1)
CH	giriş	Silindir (cylinder) numarası
CL	giriş	Sektör numarası (1'den başlar!)
DH	giriş	Kafa (head) numarası
DL	giriş	Sürücü numarası (0x80 = ilk sabit disk)
ES:BX	giriş	Okunan verinin yükleneceği bellek adresi (segment:offset)
CF	çıkış	0 = başarılı, 1 = hata
AH	çıkış	Hata durumunda hata kodu
DI	yerel	Örnekteki yeniden deneme sayacı (BIOS kullanmaz, biz kullanırız)

İpucu

Dikkat: **CX** register'ı burada tek başına bir sayı değil, *iki ayrı bilgiye* bölünmüştür – **CH** silindiri, **CL** sektörü taşır. Benzer şekilde **DX** de **DH** (kafa) ve **DL** (sürücü) olarak ikiye ayrılır. **ES:BX** çifti ise "veriyi belleğin neresine koyayım?" sorusunu yanıtlar. Örnekteki **DI** ise BIOS'a ait değildir; biz onu yeniden deneme sayacı olarak kullanırız.

16.9 INT 13h, AH=42h – LBA Okuma

Register	Yön	Rolü
AH	giriş	0x42 – "genişletilmiş okuma"
DL	giriş	Sürücü numarası (0x80)
DS:SI	giriş	Disk Adres Paketi'nin (DAP) adresi
CF	çıkış	0 = başarılı, 1 = hata

CHS'nin aksine burada tüm parametreler (sektör sayısı, hedef, LBA numarası) **DS:SI**'nin gösterdiği bellekteki bir *yapıda* durur; register'lar sadece o yapıyı işaret eder. Bu, çok sayıda parametreyi az register'la geçirmenin yaygın yöntemidir.

16.10 INT 15h, EAX=E820h – Bellek Haritası

Register	Yön	Bu örnekteki rolü
EAX	giriş/çıkış	Girişte 0xE820; her çağrıda yeniden ayarlanır. Çıkışta 'SMAP' imzası döner
EBX	giriş/çıkış	"Devam" göstergesi: girişte 0, her çağrıda bir sonraki bölge için güncellenir; 0 olunca liste biter
ECX	giriş	Bir girdinin bayt cinsinden boyutu (24)
EDX	giriş	Sabit 0x534D4150 imzası ('SMAP')
ES:DI	giriş	Bölge tanımının yazılacağı tampon adresi
CF	çıkış	1 = hata veya listenin sonu

Not

Bu, **EBX**'in "durum taşıyıcı" olarak kullanıldığı güzel bir örnektir: BIOS **EBX**'e her seferinde kaldığı yeri yazar, biz de onu değiştirmeden bir sonraki çağrıya veririz. Yani **EBX** çağrılar arasında *iterator* (yineleyici) görevi görür. **EAX** ve **ECX**'in her turda yeniden ayarlanması gerekir çünkü BIOS bunların üzerine yazar.

16.11 Diğer INT 15h Fonksiyonları

Fonksiyon	Register	Rolü
E801h	AX=0xE801	Fonksiyon seçici
	AX/CX (çıkış)	1–16 MB arası bellek (KB)
	BX/DX (çıkış)	16 MB üstü bellek (64 KB birim)
88h	AH=0x88	Fonksiyon seçici
	AX (çıkış)	1 MB üstü bellek (KB)
2401h	AX=0x2401	A20'yi aç; CF çıkışta hata durumu
86h	AH=0x86	Bekleme fonksiyonu
	CX:DX	Mikrosaniye cinsinden süre (32-bit değer iki register'a bölünür)

İpucu

CX:DX çiftinde 32-bit bir sayının nasıl taşındığına dikkat edin: **CX** üst 16 biti, **DX** alt 16 biti tutar. 16-bit modda 32-bit değer tek register'a sığmadığı için bu "register çifti" tekniği kullanılır – tıpkı çarpma sonucunun **DX:AX**'te dönmesi gibi.

16.12 INT 1Ah (Saat) ve INT 12h (Bellek)

Kesme	Register	Rolü
INT 1Ah	AH=0x00	Fonksiyon: sayacı oku
	CX:DX (çıkış)	Gün başından beri geçen tik sayısı (32-bit, çift register)
INT 12h	(giriş yok)	Parametre almaz
	AX (çıkış)	Taban bellek boyutu (KB)

INT 12h, hiç giriş parametresi almayan nadir bir kesmedir; sonucu doğrudan **AX**'te verir. INT 1Ah ise saat değerini yine bir **CX:DX** çiftinde döndürür.

16.13 Özet: Register'ların BIOS Çağrılarındaki Roller

Register	BIOS çağrılarındaki tipik rolü
AH	Neredeyse her zaman fonksiyon seçici (hangi alt işlem)
AL	Karakter, mod numarası, sektör sayısı gibi tekil parametre; sık sık çıkışta da kullanılır
BX / BH / BL	Sayfa, renk özniteliği veya hedef offset (ES:BX)
CX / CH / CL	Sayaç, tekrar sayısı veya CHS'de silindir+sektör
DX / DH / DL	Sürücü, koordinat, kafa veya CX:DX 'te 32-bit değer alt yarısı
SI / DI	Bellek işaretçisi (metin, DAP, tampon adresi)
ES / DS	BX/SI/DI ile birlikte adresin segment kısmı
CF (bayrak)	Çıkışta hata göstergesi (1 = hata)
ZF (bayrak)	Bazı çağrılarda durum çıkışı (örn. klavye tamponu boş mu)

17. Tam Örnek: İki Aşamalı Önyükleyici

Aşağıdaki tam çalışan örnek, bir önyükleyicinin tipik görevlerini bir arada gösterir: segmentleri kur, mesaj yazdır, diskten ikinci aşamayı yükle ve ona atla. Bu, gerçek işletim sistemlerinin önyükleme mantığının basitleştirilmiş hâlidir.

```

1 bits 16 ; 16-bit gercek mod
2 org 0x7C00 ; BIOS onyukleyiciyi buraya yukler
3
4 start:
5 ; --- 1. Segmentleri ve yigini guvenli sekilde ayarla ---
6 cli ; kesmeleri kapat (kurulum sirasinda)
7 xor ax, ax
8 mov ds, ax ; DS = 0

```



```

9      mov es, ax      ; ES = 0
10     mov ss, ax      ; SS = 0
11     mov sp, 0x7C00   ; yigin onyukleyicinin altinda
12     sti              ; kesmeleri geri ac
13
14     ; --- 2. Karsilama mesaji yazdir ---
15     mov si, message
16     call printString
17
18     ; --- 3. Diskten ikinci asamayi yukle (sektor 2) ---
19     mov di, 3        ; 3 deneme hakki
20 .read:
21     mov ah, 0x02     ; sektor oku
22     mov al, 1        ; 1 sektor
23     mov ch, 0        ; silindir 0
24     mov cl, 2        ; sektor 2
25     mov dh, 0        ; kafa 0
26     mov dl, 0x80     ; ilk sabit disk
27     mov bx, 0x8000   ; ES:BX = 0000:8000 hedef adres
28     int 0x13
29     jnc .loaded
30     xor ah, ah       ; hata -> diski resetle
31     int 0x13
32     dec di
33     jnz .read
34     mov si, errorMsg ; 3 deneme basarisiz -> hata mesaji
35     call printString
36     jmp halt
37
38 .loaded:
39     ; --- 4. Ikinci asamaya atla ---
40     jmp 0x0000:0x8000 ; 0x8000 adresindeki koda gec
41
42 halt:
43     hlt
44     jmp halt
45
46 ; --- Yardimci rutin: SI'daki null-terminated metni yazar ---
47 printString:
48     mov ah, 0x0E
49 .loop:
50     lodsb
51     test al, al
52     jz .end
53     int 0x10
54     jmp .loop
55 .end:
56     ret
57
58 message: db "Onyukleyici basladi, kernel yukleniyor...", 0x0D, 0x0A, 0
59 errorMsg: db "DISK HATASI!", 0x0D, 0x0A, 0
60
61 times 510-($-$$) db 0 ; 510. bayta kadar sifirla doldur
62 dw 0xAA55             ; onyukleme imzasi (son 2 bayt)

```

Önyükleme sektörü kuralları

Önyükleme sektörü **tam 512 bayt** olmalı ve son 2 baytı **0xAA55** imzasını içermelidir; BIOS bu imzayı görmezse diski önyüklenemez kabul etmez. `times 510-($-$$) db 0` ifadesi, kodun bittiği yerden 510. bayta kadar olan boşluğu sıfırla doldurur (\$ mevcut konum, \$\$ bölüm başıdır).

18. Korumalı Moda Geçiş

Önyükleyicinin son görevi, 16-bit gerçek moddan 32-bit korumalı moda geçmektir. Bunun için bir GDT (Global Descriptor Table) hazırlanır ve **CR0**'ın PE biti set edilir.

</> Gerçek moddan korumalı moda geçiş

```

1  cli                ; kesmeleri kapat (IDT henüz hazır değil)
2  lgdt [gdtDescriptor] ; GDT'yi işlemciye tanıt
3
4  mov eax, cr0
5  or  eax, 1        ; PE bitini (bit 0) set et
6  mov cr0, eax      ; artık korumalı moddayız!
7
8  ; boru hattını temizlemek için uzak atlama şart:
9  jmp 0x08:protectedMode ; 0x08 = GDT'deki kod segmenti seçici
10
11 bits 32            ; artık 32-bit kod
12 protectedMode:
13     mov ax, 0x10    ; 0x10 = veri segmenti seçici
14     mov ds, ax
15     mov es, ax
16     mov ss, ax
17     mov esp, 0x90000 ; yeni 32-bit yığın
18     ; ... 32-bit çekirdek kodu buradan devam eder ...
19
20 ; --- Global Descriptor Table ---
21 gdtStart:
22     dq 0x0000000000000000 ; null tanımlayıcı (zorunlu)
23 gdtCode:
24     dq 0x00CF9A000000FFFF ; kod segmenti (taban=0, limit=4GB)
25 gdtData:
26     dq 0x00CF92000000FFFF ; veri segmenti (taban=0, limit=4GB)
27 gdtEnd:
28
29 gdtDescriptor:
30     dw gdtEnd - gdtStart - 1 ; GDT boyutu - 1
31     dd gdtStart              ; GDT adresi

```

⚠ Dikkat

CR0'ın PE bitini set ettikten hemen sonra **uzak atlama** (far jump) yapmak zorunludur. Bu, işlemcinin komut boru hattını (pipeline) temizler ve **CS** register'ını yeni 32-bit segment seçicisiyle yükler. Bu adım atlanırsa işlemci tanımsız duruma girer.

19. Sayfalama (Paging)

Sayfalama, sanal adresleri fiziksel adreslere çeviren donanım mekanizmasıdır. Bellek 4 KB'lık "sayfa"lara bölünür; bir sayfa tablosu her sanal sayfayı bir fiziksel çerçeveye eşler. Bu sayede her programa kendi izole adres alanı verilir.

19.1 Sayfa Tablosu Hiyerarşisi

Mod	Seviye	Yapı
32-bit	2 seviye	Sayfa Dizini (PD) → Sayfa Tablosu (PT)
32-bit PAE	3 seviye	PDPT → PD → PT
64-bit	4 seviye	PML4 → PDPT → PD → PT

i Bilgi

Bir sanal adres, tablo seviyelerine bölünerek çözümlenir. 32-bit'te 32 bitlik adres şöyle ayrışır: üst 10 bit sayfa dizini indeksi, ortadaki 10 bit sayfa tablosu indeksi, alt 12 bit ise sayfa içi offset ($4\text{ KB} = 2^{12}$). İşlemci bu çeviriyi otomatik yapar; TLB (Translation Lookaside Buffer) sık kullanılan çevirileri önbelleğe alarak hızlandırır.

19.2 32-bit Sayfalamayı Etkinleştirme

Aşağıdaki örnek, ilk 4 MB'lık belleği birebir (identity mapping) eşler – yani sanal adres = fiziksel adres. Çekirdeğin kendi kodunu çalıştırmaya devam edebilmesi için bu şarttır.

</> Birebir eşlemeli sayfa tablosu kurma (32-bit)

```

1 bits 32
2 setupPaging:
3     ; --- 1. Sayfa tablosunu birebir eslemeyle doldur ---
4     mov edi, 0x101000    ; sayfa tablosunun adresi (PT)
5     mov cr3, edi        ; ileride PD adresini yazacagiz
6     mov eax, 0x00000003  ; ilk sayfa: adres=0, bayrak=Present+RW
7     mov ecx, 1024        ; 1024 girdi = 4MB kapsam
8     .fill:
9     mov [edi], eax       ; girdiyi yaz
10    add eax, 0x1000      ; sonraki 4KB sayfaya gec
11    add edi, 4           ; sonraki girdi (her girdi 4 bayt)
12    loop .fill
13
14    ; --- 2. Sayfa dizinini kur (tek girdi PT'yi gosterir) ---
15    mov edi, 0x100000    ; sayfa dizini (PD) adresi
16    mov eax, 0x101003    ; PT adresi | Present | RW
17    mov [edi], eax
18    mov dword [edi+4], 0 ; diger PD girdileri bos (Present=0)
19
20    ; --- 3. CR3'e sayfa dizini adresini yukle ---
21    mov eax, 0x100000
22    mov cr3, eax
23
24    ; --- 4. CR0'in PG bitini (bit 31) set ederek sayfalamayi ac ---
25    mov eax, cr0
26    or  eax, 0x80000000
27    mov cr0, eax        ; sayfalama artik AKTIF
28    ret

```

19.3 Sayfa Tablosu Girdisi (PTE) Bayrakları

Her sayfa tablosu girdisinin alt 12 biti kontrol bayraklarıdır (adresin alt 12 biti zaten 0 olduğu için bu bitler boştur ve bayrak olarak kullanılır).

Bit	Ad	Anlamı
0	P (Present)	Sayfa bellekte mevcut mu? 0 ise erişim page fault üretir
1	R/W	1=yazılabilir, 0=salt okunur
2	U/S	1=kullanıcı (ring 3), 0=yalnızca çekirdek (ring 0)
3	PWT	Write-through önbellekleme
4	PCD	Önbelleği devre dışı bırak
5	A (Accessed)	İşlemci erişince otomatik set edilir
6	D (Dirty)	Sayfaya yazılınca set edilir (swap için kullanılır)
63	NX	(64-bit) Bu sayfada kod çalıştırmayı yasakla

⚠ Dikkat

Sayfalamayı açtıktan sonra, o an çalışan kodun bulunduğu adres birebir eşlenmiş **olmalıdır**. Aksi halde **CR0**'a yazan komuttan sonraki komutu getirmeye çalışan işlemci geçersiz bir sanal adrese bakar ve tripla hata (triple fault) ile yeniden başlar. Bu yüzden önce identity mapping yapılır.

💡 İpucu

Bir sayfa tablosu girdisini değiştirdiğinizde TLB'deki eski çeviri hâlâ geçerli olabilir. `invlpg [adres]` komutuyla ilgili TLB girdisini geçersizleştirebilir, ya da **CR3**'ü yeniden yükleyerek tüm TLB'yi temizleyebilirsiniz.

20. Kesme İşleyici (ISR) Yazma

Şimdiye kadar BIOS'un *sağladığı* kesmeleri kullandık. Bir işletim sistemi ise kendi kesme işleyicilerini (ISR – Interrupt Service Routine) yazar. Korunmalı modda bunun için **IDT** (Interrupt Descriptor Table) kurulur.

20.1 IDT – Kesme Tanımlayıcı Tablosu

Gerçek moddaki basit IVT'nin yerini korunmalı modda IDT alır. IDT'de her girdi 8 bayttır ve bir kesme işleyicisinin adresini, segment seçicisini ve tipini tutar.

Alan	Açıklama
Offset (0–15)	İşleyici adresinin alt 16 biti
Seğici	Kod segmenti seçicisi (genellikle 0x08)
Tip & öznitelik	Kapı tipi: 0x8E =kesme kapısı (ring 0)
Offset (16–31)	İşleyici adresinin üst 16 biti

20.2 Bir IDT Girdisi Ayarlama

</> Belirli bir kesme numarasına işleyici atama rutini

```

1  bits 32
2  ; Giris: AL = kesme numarası, EBX = işleyici adresi
3  idtSetEntry:
4      push edi
5      movzx edi, al          ; EDI = kesme numarası
6      shl edi, 3            ; her girdi 8 bayt -> indeks*8
7      add edi, idt          ; IDT içindeki girdi adresi
8
9      mov [edi], bx         ; offset bit 0-15
10     mov word [edi+2], 0x08 ; kod segmenti seçicisi
11     mov byte [edi+4], 0    ; ayrılmış (0)
12     mov byte [edi+5], 0x8E ; tip: 32-bit kesme kapısı, ring 0
13     shr ebx, 16
14     mov [edi+6], bx       ; offset bit 16-31
15     pop edi
16     ret
17
18 ; IDT: 256 girdi x 8 bayt
19 idt: times 256*8 db 0
20 idtDescriptor:
21     dw 256*8 - 1          ; IDT boyutu - 1
22     dd idt                ; IDT adresi

```

20.3 Basit Bir Kesme İşleyicisi

İşleyiciler `iret` (interrupt return) ile bitmelidir; `ret` değil. İşleyici, kullandığı tüm register'ları koruyup geri yüklemelidir.

</> Ekranı nokta basan basit ISR (örneğin zamanlayıcı)

```

1  bits 32
2  timerIsr:
3      pusha                ; TUM genel register'ları sakla
4
5      ; --- işleyici gövdesi: ekranın sol üstüne '.' yaz ---
6      mov edi, 0xB8000      ; VGA metin belleğinin adresi
7      mov byte [edi], '.'   ; karakter
8      mov byte [edi+1], 0x0A ; renk: açık yeşil
9
10     ; --- PIC'e "kesme islendi" bildir (End Of Interrupt) ---
11     mov al, 0x20          ; EOI komutu
12     out 0x20, al          ; ana PIC'e gönder (port 0x20)
13
14     popa                  ; register'ları geri yükle
15     iret                  ; kesmeden dön (EFLAGS + CS:EIP geri yüklenir)

```

⚠ Dikkat

Bir donanım kesmesi işleyicisinin sonunda **PIC'e EOI (End Of Interrupt)** sinyali göndermek zorunludur (out 0x20, al ile 0x20 değeri). Aksi halde PIC o kesme hattının daha fazla kesme göndermesini engeller ve sistem kilitlenir. Slave PIC'ten (IRQ 8–15) gelen kesmelerde hem 0xA0 hem de 0x20 portlarına EOI gönderilir.

20.4 PIC'i Yeniden Haritalama

Varsayılan olarak donanım kesmeleri (IRQ 0–15), işlemcinin istisna (exception) numaralarıyla (0–31) çıkarılır. Bu yüzden PIC (8259A denetleyici) yeniden programlanarak IRQ'lar 0x20–0x2F aralığına taşınır.

⌘ PIC'i 0x20 ofsetine yeniden haritalama

```

1 bits 32
2 picRemap:
3     mov al, 0x11          ; baslatma komutu (ICW1)
4     out 0x20, al         ; ana PIC'e
5     out 0xA0, al         ; slave PIC'e
6
7     mov al, 0x20          ; ICW2: ana PIC ofseti -> IRQ0 = int 0x20
8     out 0x21, al
9     mov al, 0x28          ; slave PIC ofseti -> IRQ8 = int 0x28
10    out 0xA1, al
11
12    mov al, 0x04          ; ICW3: ana-slave baglantisi (IRQ2)
13    out 0x21, al
14    mov al, 0x02
15    out 0xA1, al
16
17    mov al, 0x01          ; ICW4: 8086 modu
18    out 0x21, al
19    out 0xA1, al
20
21    mov al, 0x00          ; maskeyi temizle: tum kesmeler acik
22    out 0x21, al
23    out 0xA1, al
24    ret

```

20.5 Klavye Kesme İşleyicisi (IRQ1)

Klavye, IRQ1 (yeniden haritaladıktan sonra INT 0x21) üretir. İşleyici, tarama kodunu port 0x60'tan okumalıdır.

</> Klavyeden tarama kodu okuyan ISR

```

1  bits 32
2  keyboardIsr:
3      pusha
4      in al, 0x60          ; klavye veri portundan tarama kodunu oku
5
6      ; AL şimdi tarama kodunu icerir.
7      ; Ust bit 1 ise tus BIRAKILDI, 0 ise BASILDI:
8      test al, 0x80
9      jnz .released
10
11     ; --- tusa basildi: burada isle ---
12     ; ornek: tarama kodunu bir tampona kaydet
13     mov [lastKey], al
14
15 .released:
16     mov al, 0x20          ; PIC'e EOI gonder
17     out 0x20, al
18     popa
19     iret
20
21 lastKey: db 0

```

i Bilgi

Kurulum sırası önemlidir: (1) IDT'yi kur ve lidt ile yükle, (2) PIC'i yeniden haritala, (3) işleyicileri IDT'ye kaydet, (4) sti ile kesmeleri aç. Bu adımlardan biri atlanırsa ilk donanım kesmesinde sistem çöker. Klavye tarama kodu ASCII *değildir*; bir çeviri tablosuyla (scancode→ASCII) dönüştürülmelidir.

21. Hızlı Referans: Register Kullanım Kalıpları

Amaç	İlgili Register'lar
Fonksiyon dönüş değeri	RAX/EAX
Döngü sayacı	RCX/ECX
Çarpma/bölme (yüksek bölüm)	RDX/EDX
String kaynak/hedef	RSI / RDI
Yığın yönetimi	RSP / RBP
BIOS kesme fonksiyon seçimi	AH
Kesme dönüş hata durumu	CF bayrağı
G/Ç port numarası	DX

21.1 System V AMD64 Çağrı Kuralı (Linux 64-bit)

Modern 64-bit Linux'ta fonksiyon parametreleri şu sırayla register'lardan geçer:

1.	2.	3.	4.	5.	6.	Dönüş
RDI	RSI	RDY	RCX	R8	R9	RAX

Bilgi

Linux syscall kuralında ise 4. parametre **RCX** yerine **R10** kullanır ve syscall numarası **RAX**'e yazılır. Ayrıca **RBX**, **EBP**, **R12–R15** "çağrı boyunca korunan" (callee-saved) register'lardır; bir fonksiyon bunları değiştirirse geri yüklemekle yükümlüdür.

64-bit Linux'ta "Merhaba" yazan syscall örneği

```

1 section .data
2     message db "Merhaba", 10 ; 10 = yeni satir
3     msgLen equ $ - message ; metnin uzunlugu
4
5 section .text
6     global _start
7     _start:
8         mov rax, 1 ; sys_write cagri numarasi
9         mov rdi, 1 ; dosya tanimlayici: stdout
10        mov rsi, message ; tampon adresi
11        mov rdx, msgLen ; yazilacak bayt sayisi
12        syscall ; cekirdegci cagir
13
14        mov rax, 60 ; sys_exit cagri numarasi
15        xor rdi, rdi ; cikis kodu 0
16        syscall

```

22. Pratik Kod Örnekleri

Bu bölüm, yukarıdaki kavramları birleştiren sık kullanılan rutinleri içerir.

22.1 String Uzunluğu (strlen)

Null ile biten bir metnin uzunluğunu sayar – C'deki strlen karşılığı.

SI'daki metnin uzunluğunu CX'te döndürme

```

1 strlen: ; giris: SI = metin, cikis: CX = uzunluk
2     xor cx, cx ; sayac = 0
3     .loop:
4         mov al, [si] ; siradaki karakter
5         test al, al ; null mu?
6         jz .end
7         inc cx ; uzunlugu artir
8         inc si ; sonraki karaktere gec
9         jmp .loop
10    .end:
11    ret

```

22.2 İki String'i Karşılaştırma (strcmp)

</> SCASB/CMPSB olmadan elle karşılaştırma

```

1 strcmp:                ; SI ve DI: iki metin. Esitse ZF=1 doner
2 .loop:
3     mov al, [si]
4     mov bl, [di]
5     cmp al, bl
6     jne .notEqual      ; karakterler farkli -> esit degil
7     test al, al         ; ikisi de null mu? (metin sonu)
8     jz .equal          ; evet -> metinler ayni
9     inc si
10    inc di
11    jmp .loop
12 .notEqual:
13     ; ZF=0 (cmp farkli oldugu icin)
14     ret
15 .equal:
16     ; ZF=1
17     ret

```

22.3 Sayıyı Onlu (Decimal) Olarak Ekrana Yazma

Bir tamsayıyı ekrana rakamlarıyla basmak, sayıyı 10'a bölerek kalanları ters sırada toplamayı gerektirir. Klasik bir alçak seviye problemi.

</> AX'teki işaretsiz sayıyı ekrana yazdırma (gerçek mod)

```

1 printNumber:           ; giris: AX = yazdirilacak sayi
2     push bx
3     push cx
4     push dx
5     mov cx, 0           ; kac rakam yiginda oldugunu say
6     mov bx, 10          ; bolen (taban 10)
7 .divide:
8     xor dx, dx          ; DX'i temizle (DX:AX / BX icin)
9     div bx              ; AX = AX/10, DX = kalan (bir rakam)
10    add dl, '0'         ; rakami ASCII'ye cevir
11    push dx             ; yigina at (ters sirada birikir)
12    inc cx
13    test ax, ax         ; sayi bitti mi?
14    jnz .divide
15 .write:
16    pop dx              ; rakamlari dogru sirada geri cek
17    mov ah, 0x0E
18    mov al, dl
19    int 0x10            ; ekrana yaz
20    loop .write
21    pop dx
22    pop cx
23    pop bx
24    ret

```

22.4 Baytı Onaltılık (Hex) Olarak Yazma

AL'deki baytı iki hex rakamı olarak yazma

```

1 printHex:                ; giris: AL = bayt
2     push ax
3     mov ah, al           ; kopyala
4     shr al, 4            ; ust yarim-bayti al (yuksekk nibble)
5     call .nibble
6     mov al, ah
7     and al, 0x0F         ; alt yarim-bayt (dusukk nibble)
8     call .nibble
9     pop ax
10    ret
11 .nibble:                ; AL'deki 0-15 degerini hex rakamina ceviri
12     cmp al, 10
13     jl .digit
14     add al, 'A' - 10     ; 10-15 -> 'A'-'F'
15     jmp .write
16 .digit:
17     add al, '0'          ; 0-9 -> '0'-'9'
18 .write:
19     mov ah, 0x0E
20     int 0x10
21     ret

```

22.5 Basit Gecikme Döngüsü (Busy-wait)

İç içe döngüyle kaba gecikme

```

1 delay:
2     mov cx, 0xFFFF      ; dis dongu sayaci
3 .outer:
4     mov dx, 0xFFFF      ; ic dongu sayaci
5 .inner:
6     dec dx
7     jnz .inner           ; ic dongu bitene kadar
8     dec cx
9     jnz .outer          ; dis dongu bitene kadar
10    ret

```

⚠ Dikkat

Busy-wait gecikmeleri işlemci hızına bağlıdır ve her makinede farklı süre alır; bu yüzden gerçek uygulamalarda INT 15h AH=86h (mikrosaniye) veya zamanlayıcı kesmesi tercih edilir. Yukarıdaki döngü yalnızca kaba testler içindir.

23. Derleme ve Test

İpucu

Önyükleyici örneklerini test etmek için NASM ile derleyip QEMU’da çalıştırabilirsiniz:

```
nasm -f bin bootloader.asm -o bootloader.bin
qemu-system-x86_64 -drive format=raw,file=bootloader.bin
```

64-bit Linux programını derleyip çalıştırmak için:

```
nasm -f elf64 program.asm -o program.o
ld program.o -o program
./program
```

Hata ayıklama için QEMU’ya `-s -S` ekleyip GDB ile `target remote localhost:1234` bağlanabilirsiniz.

24. Kapanış Notları

Bu rehber, x86 register mimarisinin temel taşlarını, adresleme modlarını, SIMD birimlerini ve gerçek modda en çok kullanılan BIOS kesmelerini bol örnekle kapsadı. Özetle:

- **Register’lar** işlemcinin en hızlı çalışma belleğidir; her birinin geleneksel bir amacı ve boyut hiyerarşisi (RAX→EAX→AX→AL) vardır.
- **Adresleme modları**, özellikle ölçekli indeks, dizilere ve yapılara verimli erişim sağlar.
- **Bayraklar** karar mekanizmasının kalbidir; CMP+koşullu atlama ikilisi tüm dallanmaların temelidir.
- **BIOS kesmeleri**, gerçek modda donanıma açılan standart kapılardır; **AH** fonksiyonu, diğer register’lar parametreleri, **CF** ise hata durumunu taşır.
- **Önyükleyici** bu parçaları birleştirir: segment kurulumu, mesaj, disk okuma ve korumalı moda geçiş.

İkisini birlikte kavramak – register’lar ve kesmeler – önyükleyici yazmaktan işletim sistemi geliştirmeye kadar tüm alçak seviye programlamanın temelidir.

Rehber sonu – İyi çalışmalar!